

使用方法

- **5-1:定義方法**
 - 何謂方法.....5-2
 - 為類別加入方法.....5-5
- **5-2:呼叫方法**
 - 呼叫同類別的方法.....5-6
 - 呼叫不同類別的方法.....5-7
- **5-3:使用return敘述/傳回執行結果**
 - 使用return敘述.....5-8
 - 傳回執行結果.....5-9
 - 使用區域變數.....5-10

Module 05

何謂方法-1

- 方法(Method) 是用來包裝特定流程的函式(Function)或程序(Process)。
- 每個方法都有獨立的名稱，必須是合法的 C# 識別字，且不可重複。
 - 同一層級範圍內，方法名稱不得與變數、常數、屬性或其他方法同名。
- 方法中宣告的變數為區域變數 (Local Variable)，僅在該方法內有效，不會與其他方法的變數衝突。
- 優點：
 - 程式碼結構更簡潔明確。
 - 提升可讀性與邏輯清晰度。
 - 方便除錯與維護。

何謂方法-2

未使用方法

```
static void Main(string[] args)
{
    int Top, Bot, Height , area;
    string result;

    Top = 10; Bot = 20; Height = 5;
    area = (Top + Bot) * Height / 2;
    result = "梯型面積：" + area;
    Console.WriteLine(result);

    Top = 9; Bot = 18; Height = 7;
    area = (Top + Bot) * Height / 2;
    result = "梯型面積：" + area;
    Console.WriteLine(result);

    Console.ReadLine();
}
```



使用方法

```
static void Main(string[] args)
{
    ShowMyArea(10, 20, 5);

    ShowMyArea(9, 18, 7);

    Console.ReadLine();
}

static void ShowMyArea(int Top , int Bot , int Height)
{
    int area = (Top + Bot) * Height / 2;
    string result = "梯型面積：" + area;
    Console.WriteLine(result);
}
```

何謂方法-3

- **C#的方法依定義可分成三大類：**
 - **.NET 提供的方法。**
 - 由 .NET Framework / .NET Core 內建提供。
 - 例如：`Console.WriteLine();`、`Math.Sqrt();` 等。
 - **自定義方法。**
 - 由開發者自行撰寫。
 - 可自行定義存取範圍、參數、回傳型態、回傳值。
 - **事件(Event)方法。**
 - 與事件機制相關，用於事件觸發時執行動作。
 - 例如按鈕點擊事件的對應方法：`button_Click(object sender, EventArgs e)`。

為類別加入方法

- 定義方法的語法：
 - 存取修飾詞 傳回值型態 方法名稱(參數串列)
{
 //...程式碼...
 return 傳回值; // void沒有傳回值
}
- 回傳值型態：
 - void方法：沒有傳回值的方法。
 - 非void方法：有傳回值的方法，必須傳回與定義型態相符的結果值。

呼叫同類別的方法

- 方法(參數串列)

```
static void Main(string[] args)
{
    ShowMyArea(10, 20, 5);

    ShowMyArea(9, 18, 7);

    Console.ReadLine();
}

static void ShowMyArea(int Top, int Bot, int Height)
{
    int area = (Top + Bot) * Height / 2;
    string result = "梯型面積：" + area;
    Console.WriteLine(result);
}
```

呼叫不同類別的方法

- 類別名稱.方法(參數串列)
- 非靜態類別必須先建立物件實體 (Instance) 才能呼叫。
- 被呼叫的方法必須注意存取型態，其存取範圍必須允許外部存取。

```
class Program
{
    static void Main(string[] args)
    {
        MyCalculate mc = new MyCalculate();
        mc.ShowMyArea(10, 20, 5);
        Console.ReadKey();
    }
}

class MyCalculate
{
    internal void ShowMyArea(int Top, int Bot, int Height)
    {
        int area = (Top + Bot) * Height / 2;
        string result = "梯型面積：" + area;
        Console.WriteLine(result);
    }
}
```

使用return敘述

- 當執行到return敘述就會立刻結束方法。
- return 之後的程式碼將不會被執行
- 非void的方法必須要有return來傳回執行結果。

```
static void ShowMyArea(int Top , int Bot , int Height)
{
    if (Top > Bot)
    {
        return;
    }
    int area = (Top + Bot) * Height / 2;
    string result = "梯型面積：" + area;
    Console.WriteLine(result);
    MyNestedMethod();
}
```


傳回執行結果

- 必須使用**return**傳回執行結果。
- **return**傳回的值，型態必須與方法定義的回傳型態相同。

```
static void Main(string[] args)
{
    int ans1 = MyArea(10, 20, 5);
    int ans2 = MyArea(9, 18, 7);

    Console.WriteLine("梯型面積：" + ans1);
    Console.WriteLine("梯型面積：" + ans2);
    Console.ReadLine();
}
```

```
static int MyArea(int Top, int Bot, int Height)
{
    int area = (Top + Bot) * Height / 2;
    return area;
}
```



使用區域變數

- 區域變數(Local Variable) :
 - 宣告在方法內部的變數。
 - 僅能在該方法中使用。
 - 當方法執行結束後，變數就會自動被釋放（消失）。
- 靜態變數(Static Variable) :
 - 被所有物件共同共享。
 - 記憶體中只存在一份資料。
 - 可直接使用「類別名稱.變數名稱」呼叫。



名稱衝突問題：

- 區域變數與靜態變數不建議使用相同名稱。
- 若名稱重複，可能導致混淆或誤用。